P2504R0
Computations as a global solution to concurrency

Published Proposal, 2021-12-11

Author:
Lucian Radu Teodorescu

Source:
GitHub

Issue Tracking:
GitHub

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:
SG1, LEWG

Table of Contents

§ 1. Introduction

This paper aims at providing proof that the senders/receivers model proposed by [P2300R2] can constitute a global solution to concurrency.

§ 1.1. Motivation

Adding a concurrency model in the C++ standard is not something to be taken lightly. It is not an incremental change, it's a major shift in direction, which hopefully changes the way C++ developers write concurrent code.

When proposing such a model, we have to ensure that we have the right semantics at the low-level (composability, error handling, efficiency, usability, genericity, interoperability with other language features, etc.), but also at the high-level -- the generality of the proposed model. As the low-level parts of the proposal are well covered by the proposal itself and other papers/discussions in the standard committee, this paper explores the generality of the proposed model.

It tries to answer the following questions:

- Can senders/receivers be used to solve any type of concurrent problem?
- Can senders/receivers eliminate the classic approach of using locks?
- Can servers/receivers be used as building blocks, while designing the concurrency aspects of the programs?
- Is there anything else that we need to add to the proposal so that senders/receivers become a general solution to concurrency?

§ 1.2. Approach and limitations

The paper tries to consider concurrent problems in a general form. It tries to prove general characteristics of programs that can be built using this model. The paper is just concerned with the fact that applications have work that needs to be executed, and they can utilize threads to perform the work. While doing so, the paper abstracts out the low-level details of the problems.

For example, the following points are not covered at all in the paper:

- error handling
- performance
- usability of the code
- any transformations that the programmer might need to do from a "classic" model to the model proposed by senders/receivers

All of these are extremely important points, but they do not constitute the focus of this paper. The paper shows that you can build concurrent programs out of computations hierarchically, but it doesn't

provide the best practices for that. This is similar to the work of Böhm and Jacopini [Böhm-Jacopini] that proves that all programs can be written with sequence, selection, and repetition, but does not show the best practices to structure the code.

This work is based on the previous work of the author [Teodorescu20a], [Teodorescu20b], [Teodorescu20c], [Teodorescu21a], [Teodorescu21b]. Parts of [Teodorescu20b] were extended and included in this paper; the ideas found in [Teodorescu21b] are also found in this paper in an more formal and explicit sense. [Teodorescu21a] shows that one can make raw tasks work with composition and decomposition; however after a more careful examination, one can see that the solution presented there is not as generic, and not as efficient as what can be done with the framework presented in [P2300R2].

§ 2. Revision history

§ 2.1. R0

Initial revision.

§ 3. Conventions and terminology

§ 3.1. Concurrent applications

This paper looks at applications from a concurrency perspective. We are interested in the design of the concurrent application, on how the work *can* be arranged onto threads, not on how it will actually be executed on the target machine.

For our discussion, we use the sufficiently general terms ***work*** and ***threads***. The work can mean any type of work that can be executed in a concurrent/parallel machine, and the thread can mean any type of execution abstraction. For example, threads can be OS threads, can be CPU cores of different types, can be GPU cores, etc.

The aim is to focus on the most general aspects of concurrency to include all classes of concurrency problems.

For exposing certain types of problems it’s often useful to assume that the target machine has an infinite amount of threads available.

§ 3.2. Tasks and computations

We call a ***task*** an independent unit of work, executed on one thread (the execution of the work on the thread is serial).

We call a ***unit of work*** that work which doesn't make sense to be divided further for concurrency reasons. If a work chunk has distinct parts that have different concurrency constraints, then that particular work is not a unit of work.

The tasks should be *independent*, in the following sense: two tasks that can be run in parallel should not block on each other.

Whenever we are referring to work and tasks, we are referring to actual run-time work. For example, if the body of a function corresponds to the unit of work, we would have as many tasks as we would have invocations of that function. Tasks describe run-time behavior, not static behavior.

We call an ***async computation*** what [P2300R2] calls a *sender object*. As the whole paper is about asynchronicity, we often omit the prefix *async* and simply call it ***computation***. The reader familiar with the *sender* should remember to substitute *computation* for *sender*.

Computations can represent parts of tasks (when multiple senders are chained together to form a task), can represent exactly one task (a computation is executed on one thread, and finishing the computation typically means finishing the work to be executed on that thread), or more than one task (when work is executed on more than one thread). We call a ***unit computation*** a computation that matches a task. Also, we call a ***multi-unit computation*** a computation that corresponds to multiple (complete) tasks.

This paper relies on the fact that for both tasks and computations one can define completion notification, which will eventually be called whenever the task/computation is finished executed (either successfully or with error, or it was canceled). However, this paper doesn't pose any restrictions on how this is actually implemented. Computations have, in general, stronger guarantees than tasks with respect to completion notifications; this is important to the constructions we make here. But then again, for the sake of simplicity, we assume that tasks can have (one way or another) the same guarantees that a completion notification is called whenever the task is finished executing.

Tasks are more appropriate to express constraints and prove the correctness of concurrent programs, while computations (a.k.a., senders) are more appropriate to express concurrency building blocks at different levels. Having defined unit computations allows us to transfer the findings that we have on tasks to computations.

§ 3.3. Task relations

Let be the set of all the tasks in the application. Let be the set of all the sets of tasks that can run in parallel without safety issues. If , we are interested in finding what tasks we can add to such that the resulting set is also an element of .

When a task is missing from a particular safe set, we say that it has a ***conflict*** with that set. We note this mathematically as .

Most of the time, these conflicts are binary; that is, a task cannot be run in parallel with another task . We will look at the binary conflicts in more detail. But, there are cases in which conflicts are non-binary. For example, trying to simulate with tasks a semaphore with a count of 3, we may not have conflicts between any task pairs, but we might have conflicts when adding more than 3 tasks.

If is a task, we denote by the set of all the tasks of the application that are safe to run in parallel with task (we are looking at binary conflicts).

There are cases in which, by the construction of the application, a task is always run before another task . We say that is a ***predecessor*** of , and we denote this by . We also say that is a ***successor*** of . Please note that these relations can be direct, or indirect (i.e., there is a for which and).

We also denote by the set of all tasks for which , and by the set of all tasks for which .

Finally, we denote by all the tasks in the application different from , that are not in . If , we write and we say that there is a ***restriction*** between and . Please note that restriction is a reflexive relation.

We call a ***constraint*** to be either a restriction or a successor / predecessor relation. In our terminology, constraint is a binary relation, while conflict is an n-nary relation.

With these definitions, for any task the following hold:

In plain English, for a given task , all the other tasks are either predecessors, either successors, either in or in . We partition the space of all the other tasks into predecessors, successors, restricted tasks, and tasks that can safely be run in parallel. Again, these just cover binary conflicts.

§ 4. Concurrency with tasks

In this section, we show that there is a general algorithm for scheduling tasks that for all programs expressed in terms of tasks will ensure safety (with respect to concurrency) without needing user-level synchronization primitives (mutexes, semaphores, etc.). Moreover, this algorithm will have the maximum efficiency possible (under certain generalizing assumptions).

§ 4.1. All concurrent programs can be expressed in terms of tasks

One important precondition for all our claims is that concurrent programs can be represented using tasks. We don't have all the basic terms rigorously defined, so it would be hard to have rigorous proof; instead, we will attempt an informal proof.

We start from the fact that all concurrent programs can be represented with threads (of various types) and chunks of work associated with threads.

The chunks of work associated with threads are typically bigger than what we call units of work. In this case, we break down larger chunks of work into smaller chunks of work until we reach units of work, i.e., tasks.

There are cases in which chunks of work contain synchronization primitives. We break this into smaller chunks of work, at the points in which the synchronization points interact with the execution (i.e., at the points in which the synchronization primitives are called). While doing so, we replace the effect of synchronization primitives with constraints.

There are ways to transform all the synchronization primitives in constraints/conflicts between work chunks. We will exemplify this with the transformation of a (locally scoped) mutex into constraints

(partially because mutexes are very common, and partially because we can use locally scoped mutexes to implement all other synchronization items).

Chunks of work containing a particular locally scoped mutex can be expressed as , where is the work done before taking the mutex, is the work done while holding the mutex, and the work done after exiting the mutex. With this, one needs to add the following constraints between the work chunks:

The reader will easily see that using these constraints we arrange the work to have exactly the same concurrent behavior.

Similarly, for semaphores, we break down the work at the points in which the semaphores are called and ensure that the work chunks that can wait have constraints relating them to the work chunks waited on. Similarly, with every synchronization primitive we break the work at the point where the synchronization primitive is called and add constraints between the parts that can block and the parts that we are blocking on.

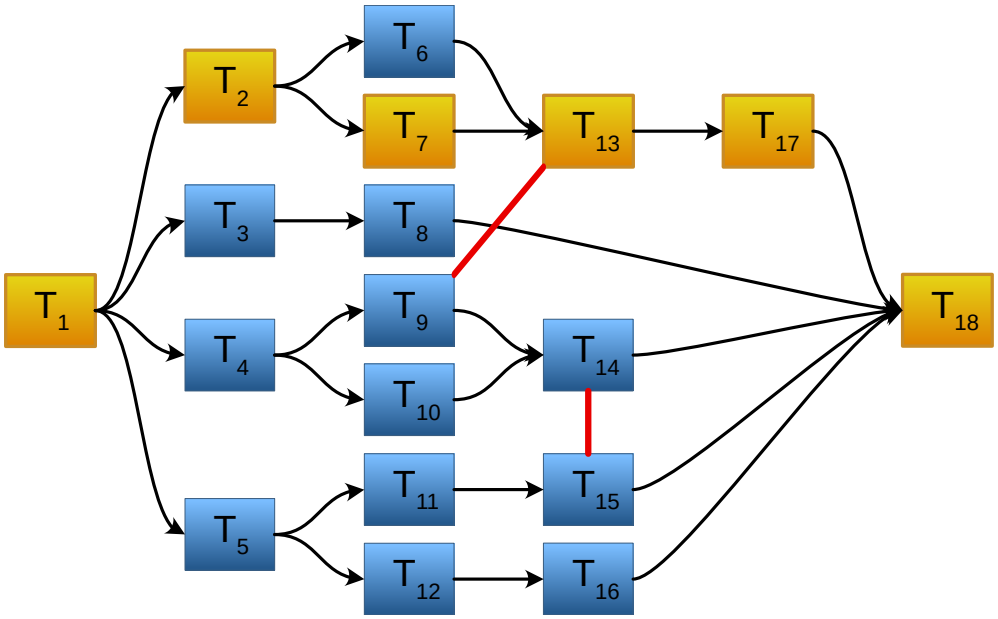
Creating threads of execution can be described with predecessor relations, where more than one chunks of work depend on one chunk of work. Similarly, thread join operations can be described with predecessor, where one chunk of work depends on multiple chunks of work (current thread and the thread we are joining with).

Thus, putting all these together, regardless of the size of the work chunks, we can divide it into units of work that are independent of each other. This leads to the conclusion that we can divide all work chunks into tasks.

If all work chunks can be divided into tasks, it means that all the programs can be represented in terms of tasks.

In other words, all the concurrent programs can be expressed as acyclic directed graphs in which we have successor relations as links, but we additionally have restrictions and general conflicts expressed in them.

The following picture shows an example of such a graph. We have arrows to represent successor / predecessor relations and simple red lines to represent restriction relations (we are showing just binary relations).



In this picture, tasks , , and are predecessors of , and task is a successor. Task has one restriction: . This means that tasks , , , , , , , , , and can be safely executed in parallel with .

We see how the program can be represented in terms of tasks and how the relations avoid the need for synchronization primitives.

This is a very important result, as it allows us to reason about the concurrency aspects of the programs and allows us to avoid blocking synchronization primitives.

§ 4.2. A general scheduling method

With respect to the lifetime of the tasks, we recognize three important moments:

- when the task is created
- when the task is ready to be executed
- when the task is finished executing

Based on these three moments, we can associate a state for each task:

- *pending* -- the task is created, but not yet ready to be executed

- *active* -- the task is ready to be executed, or in the process of executing
- *completed* -- the task has finished executing

A task can have only one state at a given time. A task is allowed to transition from *pending* to *active* and from *active* to *completed*, and no other transitions are valid.

The goal of our scheduling method is to associate move tasks from the *pending* to the *completed* state.

Please note that there might be tasks in the *active* state that are not yet executing. For example, we might have 1000 *active* tasks, but only 8 threads handling tasks; thus, we would only execute 8 tasks at a given time. Another way to look at this is to consider that the hardware potentially has an infinity of cores.

The key point of the method is to act at two different points in the lifetime of the task: when the task is created (passed to our abstraction), and when the task finished executing.

Below there is a pseudo-code for the proposed method:

```
struct safe_executor {
    void execute(task t) {
        // For simplicity, assume that everything is synchronized
        std::lock_guard<std::mutex> lock(transition_mutex());
        // Can the task be moved directly into active state?
        if ( can_be_active(t) ) {
            set_active(t);
            active_tasks().add(t);
            start_executing(t, &on_task_completed);
        }
        else {
            // Cannot execute the task yet; make it pending
            set_pending(t);
            pending_tasks().add(t);
        }
    }

private:
    void on_task_completed(task t) {
        // Atomically move tasks to pending to ensure safety
        std::lock_guard<std::mutex> lock(transition_mutex());
        set_completed(t);
        // Move to pending all the tasks that can be active
        for ( auto t2: pending_tasks() ) {
```

```
            if ( can_be_active(t2) ) {
                set_active(t2);
                start_executing(t2, &on_task_completed);
            }
        }
    }
    bool can_be_active(task& t) {
        // If not all predecessors in the completed state the task cannot b
        for ( auto t2: direct_predecessor_of(t) )
            if ( !is_completed(t2) )
                return false;
        // A task that has restrictions with another active task cannot be
        for ( auto t2: restrictions_of(t) )
            if ( is_active(t2) )
                return false;
        // We might have non-binary conflicts
        if ( has_other_conflicts(t, active_tasks()) )
            return false;
        // Otherwise the task can be active
        return true;
    }
    ...
};
```

When the task is created (i.e., added to our abstraction), we check if the task can be *active*. If yes, set it to the *active* state and start executing it. If not, the task is *pending* and add it to a list of *pending* tasks.

When a task finishes executing, it may allow other *pending* tasks to become active. We check the list of *pending* tasks and move to *active* all the possible tasks (greedy implementation).

A task can become active when all the direct predecessors are in the *completed* state and it is not a restriction of an already *active* task, and it doesn't have non-binary conflicts with the active set.

The way this method is expressed, it may be inefficient to run these functions. We expressed it like that to have a variant for the most general form and to be easy to reason about it. There are ways to make this far more efficient, but these are dependent on the problem domain. We will show some examples later.

§ 4.3. Properties of the general scheduling method

In the discussion that follows we assume that the relations between the tasks are properly constructed. If one task creates another task, we assume there is a successor/predecessor relation between them. We also assume that all the predecessors are created before their successors. We also assume that there are no circular dependencies. Also, each task must be part of a least one set from .

✧ ✧ ✧

Lemma 1 (completeness). If a concurrent program has a finite set of tasks then the general scheduling method will eventually complete executing all the tasks.

All tasks that are marked as *active* will start executing and will eventually complete.

Any concurrent program must start with at least one *active* task.

All the tasks of the program, when created, are marked either *active* or *pending*.

If we prove that all the *pending* tasks will eventually become *active*, then our method will execute all the tasks and eventually complete them.

A task is marked as *pending* because at the point of creation we had:

1. either a direct predecessor that is not complete; , where is either *active* or *pending*
2. either it has a restriction relation to one of the active tasks; , where is active
3. either , where is the current set of active tasks

When a task is marked as *completed*, we check if any of the *pending* tasks can be marked as *active*. If we converting *pending* tasks to *active* we are making progress towards our goal. But, it may happen that after completing tasks we cannot convert any *pending* tasks to be *active*. We have to prove that eventually this will not happen and we manage to convert *pending* tasks to *active*.

If the conversion doesn't happen, then the set of *active* tasks will decrease each time a task is finished executing. If no new *active* tasks are added, then eventually the set of *active* tasks will become empty.

If there is no *active* task, then it's clear the conditions 2. and 3. cannot prevent a task from moving from *pending* to *active*. Let's also show that neither condition 1. doesn't prevent converting a *pending* task to *active*. If the set of *active* tasks is empty, it means that all the predecessors discussed in rule 1. must be *pending*. Because the predecessor relation is not reflexive and is not circular, it means that eventually we can find a *pending* task that doesn't have its direct predecessors in the set of *pending*

tasks; as the set of *active* tasks is empty, it means that all the direct predecessors are in the *complete* state. Thus, that task cannot be prevented by any predecessor to become *active*.

Thus, we found at least one task that can transition from *pending* to *active*.

If we are making progress and converting *pending* tasks to *active* tasks, then we eventually convert all the *pending* tasks to *active*. And, as all the *active* tasks will complete eventually, all the tasks in our program will eventually complete.

Q.E.D.

✧ ✧ ✧

Lemma 2 (soundness). If the task relations are properly set, then the general scheduling method will not allow running in parallel tasks that can have race conditions.

Having the task relations properly set means that each time we can have race conditions between a set of tasks, we must have defined conflicts between these tasks. As tasks are units of work, which don't make sense to be divided anymore, we can't have a situation in which parts of the tasks are in conflict with other parts of other tasks; if that would have been the case, then the tasks would not be units of work.

By the construction of our general method, we can only run in parallel tasks that are at the same time in the *active* state. Also, by the construction of the method (see the mutex) we move tasks to the *active* state one at a time. The only way in which this lemma can be false is if, given a set of *active* tasks, we make *active* a new task that has conflicts with the existing set of *active* tasks. But, before we transition a task to *active* state, we check if it can have conflicts with the already *active* tasks. As this is an atomic operation, we are guaranteed that there are no conflicts between the newly added task and the existing list of active tasks.

As it is impossible to have conflicting tasks being *active* at the same time, we cannot have parallel tasks that generate race conditions.

Q.E.D.

✧ ✧ ✧

Lemma 3 (efficiency). On a machine with almost infinite parallelism, using the greedy assumption, at any time during the execution of the tasks (ignoring the time we are actually spending in the scheduling of the tasks) we cannot add any more task to be executed while maintaining soundness.

By greedy assumption, we mean that adding a task to be executed as soon as possible is a good choice. We cannot know what tasks would soon be available for execution, so, trying to hold back tasks is not necessarily a good strategy. Using the greedy assumption typically leads to good efficiency, but it is not guaranteed to have the maximum efficiency.

Let us take the set to mean all the tasks active at a given point. By convention, we don't want to consider the time spent scheduling, so we only care about moments in which any of the threads are not executing `safe_executor::execute()` nor `safe_executor::on_task_completed()`.

Let's assume that at that we can execute one more task to increase efficiency. That is, the task was in *pending* state and can move it to *active* state (by our definition of task states). To maintain soundness, we should not have a conflict between and . That is, calling `can_be_active(t)` should return `true` at that instant. As the task is currently not in *active* state it means that `can_be_active(t)` returned `false` last time it was called.

The last time that `can_be_active(t)` was called was when the task was created or when another task was completed. At that point, this function call returned `false`, as the task is still in *pending* state. Let's note by the set of *active* tasks at this point.

From that point until the point in which this task can be *active* we can have other tasks created. In that case, the set of *active* tasks can only be increased ( ). But if task was not safe to run in parallel with the tasks in , then they cannot be safe running in parallel with the same tasks and some other new tasks.

Thus, there is no way that we can execute additional tasks without compromising safety.

Q.E.D.

✧ ✧ ✧

Theorem 1 (general solution for scheduling tasks). There is a general method of scheduling tasks that works with all concurrent problems, that is safe (without the need of additional blocking synchronization), and is as efficient as it can be (under the greedy assumption).

The proof follows immediately from the above lemmas.

Q.E.D.

§ 4.4. Specialized schedulers

The general scheduling method is a global solution, but it may be too heavy for some applications. And, depending on the type of application, we might find more efficient scheduling methods. We describe here several specialized scheduling methods for such purpose.

First, let us note that most of the applications don't have conflicts between all types of tasks; we can typically partition the space of conflicts. Taking mutexes as an example, in most applications we don't have just one mutex for everything, but we have multiple mutexes; that is, we partition the space of possible conflicts. Having localized conflicts allows us to independently manage the conflicts that can appear between groups of tasks. We can use different scheduling methods in different parts of the problem.

We will now briefly consider several types of schedulers inspired by existing practice.

§ 4.4.1. Mutexes can be modeled with serializers

A mutex ensures that multiple regions of code cannot be run in parallel; a maximum of one of the marked regions can run in parallel. If we turn this into tasks, we have certain tasks that have restrictions between them. If is a set of tasks that need to be mutually exclusive, then we should add restrictions between all the tasks of this set: .

This can be modeled in the following way (pseudo-code):

```
struct serializer {
    void execute(task t) {
        atomically-do {
            if ( !executing_ )
                start_executing(t, &on_task_completed);
            else
                queued_tasks_.push(t);
            executing_ = true;
        }
    }

private:
    void on_task_completed(task t) {
        atomically-do {
            task next;
            if ( queued_tasks_.try_pop(next) )
```

```
                start_executing(next, &on_task_completed);
            else
                executing_ = false;
        }
    }

    concurrent_queue<task> queued_tasks_;
    bool executing_{false};
};
```

A real implementation for this would arrange slightly different the data structure to ensure that the operations present can be executed atomically. The important point here is that the implementation is not necessarily complicated and without significant loss of performance.

Using these abstractions instead of mutexes can benefit concurrent applications, both in terms of safety and in terms of performance. If we have plenty of other tasks in the system, the performance of the application doesn't drop when using serializers; we are not blocking any threads from executing work.

4.4.2. Semaphores can be modeled with n-serializers

Semaphores can be thought of as extensions to mutexes, in the sense that they bound the number of threads executing a protected region to a number greater than one.

Similar to the serializer abstraction, we can introduce an n-serializer abstraction that allows maximum *N* tasks to be executed at once.

If is the set of tasks for which we can execute in parallel maximum *N*, then we can define conflicts between sets of tasks. if the size of the set is greater or equal to *N*.

The reader should note that these types of conflicts cannot be represented only with binary relations. Binary relations can be used for representing the conflicts in a lot of cases, but this is a great example of their limitations.

Below there is a pseudo-code for a possible implementation:

```
struct n_serializer {
    n_serializer(int max_count) : max_count_{max_count} {}

    void execute(task t) {
```

```
        atomically-do {
            if ( num_executing_ < max_count_ ) {
                start_executing(t, &on_task_completed);
                num_executing_++;
            }
            else
                queued_tasks_.push(t);
        }
    }

private:
    void on_task_completed(task t) {
        atomically-do {
            task next;
            if ( queued_tasks_.try_pop(next) )
                start_executing(next, &on_task_completed);
            else
                num_executing_--;
        }
    }

    int max_count_;
    concurrent_queue<task> queued_tasks_;
    int num_executing_{0};
};
```

§ 4.4.3. Read-write mutexes can be modeled with rw-serializers

A read-write mutex has two types of zones: *read* zones and *write* zones. Multiple *read* zones can be executed in parallel with each other, but one cannot execute multiple *write* zones in parallel, and cannot execute a *write* zone in parallel with a *read* zone.

We can also translate this idea into the world of tasks. If is a set of *read* tasks and a set of *write* tasks, then we add the following restrictions: , and . We don't need to add any restrictions between *read* tasks.

A possible pseudo-code implementation for such a rw-serializer is given below:

```
struct rw_serializer {
    void execute_read(task t) {
        atomically-do {
            if ( !executing_write_ && queued_write_tasks_.empty() ) {
                // execute the task if we are not executing write and we do
                start_executing(t, &on_task_completed_r);
                num_executing_read_++;
            }
            else
                queued_read_tasks_.push(t);
        }
    }

    void execute_write(task t) {
        atomically-do {
            if ( !executing_write_ && !executing_read_ ) {
                // execute the task if we are not executing anything else
                start_executing(t, &on_task_completed_w);
                executing_write_ = true;
            }
            else
                queued_write_tasks_.push(t);
        }
    }

private:
    void on_task_completed_r(task t) {
        atomically-do {
            num_executing_read_--;
            start_next();
        }
    }
    void on_task_completed_w(task t) {
        atomically-do {
            executing_write_ = false;
            start_next();
        }
    }
    void start_next() {
        task next;
        if ( num_executing_read_ == 0 && queued_write_tasks_.try_pop(next)
```

```
            // If no readers are executing, try execute a write task
            start_executing(next, &on_task_completed_w);
            executing_write_ = true;
        }
        else if ( queued_write_tasks_.empty() && queued_read_tasks_.try_pop
            // If no writers are queue, try execute a read task
            start_executing(next, &on_task_completed_r);
            num_executing_read_++;
        }
    }

    concurrent_queue<task> queued_read_tasks_;
    concurrent_queue<task> queued_write_tasks_;
    int num_executing_read_{0};
    bool executing_write_{false};
};
```

There are multiple ways to implement read-write mutexes (favoring *writes*, favoring *reads*, maintaining fairness, etc.). Similarly, there are multiple ways in which one can implement a rw-serializer. The point is that we can translate all these mutex-based structures into abstractions that operate on tasks and restrictions/conflicts between them.

§ 4.4.4. Beyond serializers

The point of showing these specialized schedulers was to show that the synchronization methods that we have in our legacy programs can translate to abstractions that operate on tasks. Serializers aren't the only structures that can be created to ensure safety in concurrent applications. And all these can be implemented in an efficient matter, both in terms of maximizing the throughput of executing tasks and minimizing the time one needs to spend in scheduling tasks.

For example, one can easily create structures for executing graphs of tasks. For a particular task, one needs to keep track of the count of direct predecessors and the list of direct successors of tasks. Once all these tasks are complete (the predecessor count decreases to zero), the task can be executed. When a task completes it can decrement the counter for all its direct successors; if one successor reaches with count zero, it starts the successor task.

Another example of a common concurrent structure that one might implement is a *pipeline*. The users can define a set of stages of processing that need to be executed sequentially, and the desired parallelism for each stage, and then push data through the pipeline. When using a pipeline abstraction, the user will typically define the stages of processing not as tasks, but as *factories of tasks*. While the

processing stage remains the same, there will be multiple tasks created as multiple items flow through the pipeline.

Related to pipelines, a general approach to concurrency would be reactive programming. There we have *streams* of items flowing through different processing units (filters, joins, splits, etc.). Again, the concurrency is expressed in terms of the processing units, which need to be factories of tasks, not tasks *per se*. Tasks can be used in the underlying implementation of the reactive framework.

All these come to strengthen the idea that we can build concurrent programs on top of tasks. Moreover, we can build concurrency abstractions based on the few properties that tasks have.

§ 5. Concurrency with computations

Now that we discussed how we can build concurrent programs using tasks, we can finally move to computations and extend the results we've obtained with tasks. Again, we use the term computation to what [P2300R2] calls a *sender object*.

§ 5.1. General applicability of computations

Theorem 2 (general applicability of computations). All concurrent programs can be expressed with computations in a safe manner, without requiring blocking synchronization and with high efficiency for computation execution (under the greedy assumption).

Based on the discussion in § 3.2 Tasks and computations, every concurrent program or part of a program that we express with tasks can also be expressed with computations. If we put this together with Theorem 1 (general solution for scheduling tasks), we arrive at the conclusion of this theorem.

Q.E.D.

In other words, computations (a.k.a., senders) can be used to solve any concurrent problem.

§ 5.2. High-level concurrency

So far we discussed units of work. The question that we are trying to answer in this section is whether one can use computations to express larger chunks of work. Allowing users to operate on larger chunks of work, will raise the abstraction level that users can operate while dealing with concurrency. The end goal is to allow users to express concurrency at all abstraction levels: from high-level to low-level.

To make it easier for the reader to follow, we repeat the pseudo-code presented in § 4.2 A general scheduling method here renaming some of the functions. Please note that their structure is identical.

```
struct safe_scheduler {
    void schedule(computation c) {
        // For simplicity, assume that everything is synchronized
        std::lock_guard<std::mutex> lock(transition_mutex());
        // Can the computation be moved directly into active state?
        if ( can_be_active(c) ) {
            set_active(c);
            active_computations().add(c);
            start_executing(c, &on_computation_finished);
        }
        else {
            // Cannot execute the computation yet; make it pending
            set_pending(c);
            pending_computations().add(c);
        }
    }

private:
    void on_computation_finished(computation c) {
        // Atomically move computations to pending to ensure safety
        std::lock_guard<std::mutex> lock(transition_mutex());
        set_completed(c);
        // Move to pending all the computations that can be active
        for ( auto c2: pending_computations() ) {
            if ( can_be_active(c2) ) {
                set_active(c2);
                start_executing(c2, &on_computation_finished);
            }
        }
    }
    bool can_be_active(computation& c) {
        // If not all predecessors in the completed state the computation c
        for ( auto c2: direct_predecessor_of(t) )
            if ( !is_completed(c2) )
                return false;
        // A computation that has restrictions with another active computat
        for ( auto c2: restrictions_of(t) )
            if ( is_active(c2) )
                return false;
        // We might have non-binary conflicts
```

```
        if ( has_other_conflicts(t, active_computations()) )
            return false;
        // Otherwise the computation can be active
        return true;
    }
    ...
};
```

The above code looks slightly different than how one would write code in the spirit of [P2300R2]. However, the reader should imagine as if the above code was written in terms of senders and receivers. Again, the intent of the paper is to discuss the general properties of using computations, not necessarily insist on the actual C++ code that needs to be written.

✧ ✧ ✧

Lemma 4 (async completion). All the properties of the general scheduling method and all its implications are still true if the completion notification for a task is sent on a different thread than the thread that actually executes the task.

Analyzing the general scheduling method described in § 4.2 A general scheduling method there isn't any single condition that is placed by the completion notification (`safe_executor::on_task_completed()`). The method still works regardless of the thread that sends the completion notification. Changing the thread onto which the notifications are made, doesn't affect the validity of Theorem 1 (general solution for scheduling tasks) and thus of Theorem 2 (general applicability of computations).

Q.E.D.

This lemma proves that we can use multi-unit computations instead of unit computations (or tasks), and we can still guarantee the completeness of the program.

✧ ✧ ✧

Lemma 5 (delayed completion). If a task completion notification is delayed, as long as it will eventually be called, Lemma 1 (completeness) and Lemma 2 (soundness) are still valid.

Both Lemma 1 (completeness) and Lemma 2 (soundness) do not consider the duration of the tasks and the timing of the completion notification. They just rely on the fact that the completion notification is eventually invoked. Thus, their results are still true if the completion signal is delayed.

Q.E.D.

✧ ✧ ✧

Lemma 6 (efficiency with computations). Treating computations as atomic entities that we don't want to subdivide, then the general method of execution will ensure the maximum amount of computations can be run in parallel at any time (except time spent in scheduling), under the greedy assumption.

We won't give the full proof here; it follows the same steps as Lemma 3 (efficiency) but essentially replacing tasks with computations. The reader can follow the same steps with the new pseudo-code provided in this section to check that one can reach the same conclusions.

Q.E.D.

✧ ✧ ✧

Theorem 3 (general solution with computations). There is a general method of scheduling computations that works with all concurrent problems, that is safe (without the need of additional blocking synchronization), and is as efficient as it can be (under the greedy assumption).

The proof follows immediately from the above lemmas, and from the translation of the general method for tasks into a method for arbitrarily sized computations.

Q.E.D.

✧ ✧ ✧

In this section, we proved that one can use computations (a.k.a., senders) of arbitrary sizes (as small as tasks, or bigger than tasks) to implement all concurrent problems. With an appropriate framework, this can be done without requiring locks in user code -- this can lead to safer concurrent programs. Also, in broad terms, under the greedy assumption, and under the conflicts imposed on the system, the execution of such programs is as efficient as it can be.

This, by itself, is an important result that we obtained for computations. Still, there are a few more important results that we can arrive at for computations.

§ 5.3. Composability and decomposability

Computations have the nice property of being highly composable. Even the smallest examples with computation invoke some kind of composition. All the sender adaptor algorithms take as arguments at least one sender object (*computation* as we named it). The resulting senders are senders that contain other senders.

This indicates that one can build complex computations out of simpler computations. Let us analyze the extent of this.

✧ ✧ ✧

Lemma 7 (composition). If a program (or sub-program) can be expressed in terms of smaller-sized computations , ..., , then the whole can be expressed as a computation.

The program can have one or multiple exit points (i.e., it terminates with one or multiple receivers). If the program has multiple exit points, we can reduce that to one exit point by applying a *join* operation; this can be implemented by adding an `execution::when_all()`. After adding this extra reduction, the program can signal its completion through only one receiver.

As we proved above, we can apply the general scheduling method (on computations) to ensure that the computations , ..., (plus, if needed, the extra reduction) are properly run (safely and efficiently). As this method can be started when the first computation is started, and it ends with a channel that can be a receiver, we can encode the whole thing as a computation.

Q.E.D.

Please note that in practice we don't have to use the general method; for most of the programs we can use simpler constructs to represent relations between the computations. For example, one can use `execution::let_value()` for sequential composition and `split()` for forking and `when_all()` for joining various computations.

✧ ✧ ✧

Lemma 8 (whole program as computation). Any concurrent program can be represented by one computation.

Theorem 2 (general applicability of computations) tells us that any program can be represented using computations. Plugging this into Lemma 7 (composition) we reach the conclusion that any program

can be represented as one computation.

Q.E.D.

Using the [P2300R2] terminology, any program can be represented by one sender object.

✧ ✧ ✧

Lemma 9 (program parts as computations). If one can divide a concurrent program into parts in such a way that no task belongs to more than one such part, then any of these parts can be represented as computations.

In the condition of the lemma we assumed that parts are greater than tasks. This is to ensure that the global scheduling algorithm is properly defined. Once we have this out of our way, then however the part is defined, as it can only consist of one or multiple tasks, one can represent it in terms of one or more computations. This means that the part itself can be represented as a computation (Lemma 7 (composition)).

Q.E.D.

Please note that the relationship between tasks can be lost for arbitrary partitions of the programs into parts. We will have an example later from which one can see that trying to keep certain relations between tasks forces us to make the divisions in certain ways.

✧ ✧ ✧

Theorem 4 (general top-down decomposition). Any concurrent program can be modeled as a computation and can be decomposed in terms of computations; if those computations are large enough, they can be further decomposed until reaching unit computations.

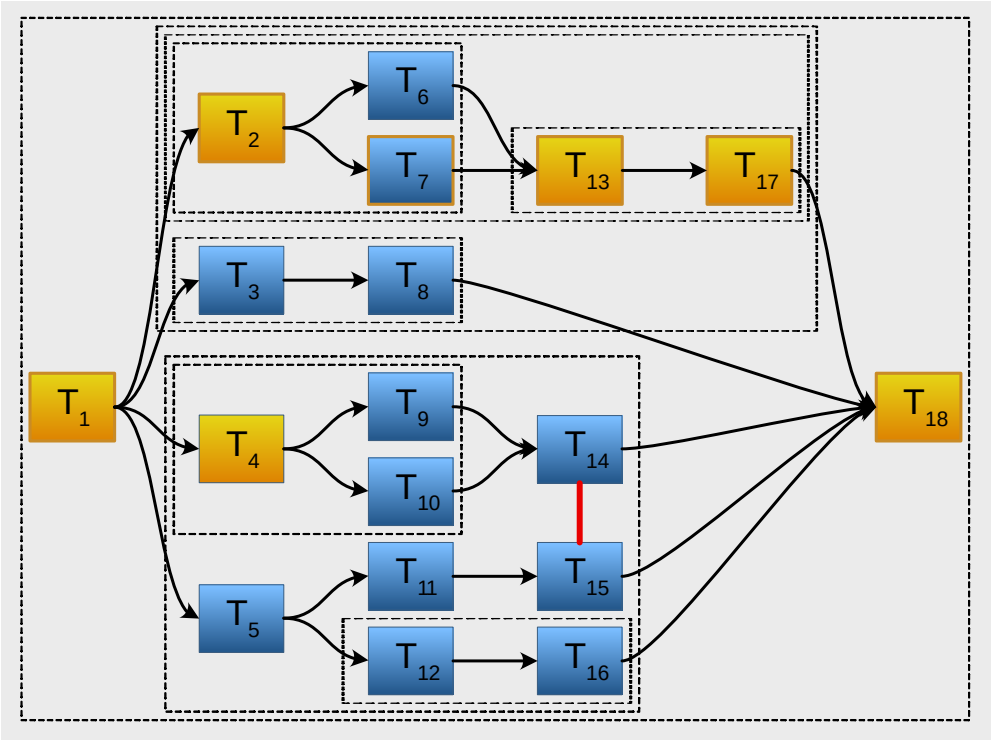
The proof follows immediately from the last 3 lemmas.

Q.E.D.

Similar to the above note, the reader should realize that this theorem doesn't state that all possible decompositions of a program or sub-program are equivalent. In the vast majority of programs, there are conflicts between too many different parts of the program. Dividing the program in different ways will result in a different set of conflicts between the computations representing the groups. This means that the performance of the program can vary depending on the way in which the split is done.

✧ ✧ ✧

Let us take an example to illustrate the applicability of Theorem 4 (general top-down decomposition). Let's consider the program illustrated in the next figure. This program contains mostly successor/predecessor relations between tasks, and one restriction (shown in red).



We consider one particular grouping in this program, and we represent with dotted rectangles these groups. Each such group can be encoded as a computation. First, the whole program can be a computation. Then, we have two major computations: one that covers the upper part, and one that can handle the bottom part. Then, each of these can be sub-divided further; we stop when we reach the task level.

To maintain the safety of the program, we need to maintain the same restrictions. The reader should note that each computation has clear predecessors, and exactly one successor.

The reader might have noticed that we did not create too much grouping around . If we add more grouping than the one shown in the picture, we need to degrade efficiency to maintain safety. For example, trying to group together and will force us to add a restriction between this group and (as stated in Lemma 6 (efficiency with computations), we treat computation atomically). Thus, even if this new group is spending a lot of time executing , we cannot execute at the same time. This leads to a loss of efficiency.

✧ ✧ ✧

This result allows us to approach concurrency in both a top-down and bottom-up fashion, allowing us to treat concurrency similarly to how we design software. That is, using computations (a.k.a., senders) we can have a structured approach to building concurrent software.

§ 5.4. Other considerations

§ 5.4.1. Using specialized schedulers

In section § 4.4 Specialized schedulers we discussed how we can replace the general scheduling method with specialized schedulers to improve the efficiency of the scheduling itself. As main examples, we've shown serializers.

Serializers can also be made to work with computations. For example, one can enqueue a lot of computations into a (simple) serializer, and the serializer will eventually execute all the computations, one at a time.

To be able to serve computations coming from different parts of the system, the serializer will probably want to store the computations in a type-erased way. Currently [P2300R2] doesn't offer a standard way of doing type-erasure on the computation. However, it will not be hard for the implementer of the serializer to do it.

Also, related to serializers, an interesting point to explore is the way in which the computations are started. Computations (i.e., senders), as defined by [P2300R2] most often have the starting point embedded inside them (senders are built on the sender obtained from the scheduler). If we want the computations to be executed one after another, to prevent unnecessary scheduler switches, maybe a good way is to reuse the scheduler from the previous computation. This is again something that can easily be done. The application probably needs some schedulers that try to reuse the previous scheduler, within certain bounds.

§ 5.4.2. Repeatable computations

While discussing tasks, the underlying assumption was that a unit of work will be mapped exactly to one task. If we have two units of work in the program, that are doing exactly the same thing, then we would also have two tasks.

The same assumption was carried over to computations. If the program needed one computation, we assumed there will be a C++ `sender` object created for it. If we execute the same work multiple

times, we need to have multiple C++ objects.

This was fine for our proofs, but it is not clear how computations can be used when the same type of work needs to be executed multiple times.

Let's take two examples: a pipeline and using reactive programming.

For pipelines, one will specify a set of *stages* that need to be sequentially applied to the items flowing through the pipeline and the concurrency constraints that apply to each of the stages. For example, we might consider a 3-stage pipeline in which the first and the last stage needs to be serialized (only one item flowing), while in the middle stage we can have multiple items being processed at the same time.

The stages of the pipeline can be expressed using computations, but again, computations cannot be reused. Thus, if items flow through a 3-stage pipeline, we need to have actual computation objects. Thus, whenever we define the pipeline we need to define the *blueprints* of these computations, and not the computations themselves. There are multiple ways of solving this problem. As an example, one can pass computation factories to the pipeline when specifying a stage. As another example, the implementer of the pipeline can decide to use prototype computations; have one computation object that is copied each time an actual item needs to be processed.

Let us switch now to reactive programming. In this model, one can define *streams* of events, and one can add *filters*, *transforms* and other operations that take streams as inputs and generate streams as outputs. As this is a bit abstract, here is an example of how these can be coded in C++ (without an apparent use of computations):

```
stream<mouse_event> auto all_mouse_events = ...
stream<mouse_event> auto mouse_moves = filter(all_mouse_events, &only_mouse
stream<mouse_event> auto relative_moves = map(mouse_moves, &offset_to_windo
stream<draw_state> auto drawing_states = ...
auto drawing_events = when_any(drawing_states, relative_moves);
drawing_events.then(&draw_on_window);
```

In this example, we passed functions to the stream algorithms, for simplicity. But we can pass computations as well. We can build reactive programming with the same technique, but for that, we need to apply the same technique of generating a lot of computations from blueprints.

Let us sketch an example for an HTTP server:

```
stream<buffer> auto packets_stream = ...
sender auto recognize_request = ... // moves computation to other thread
stream<http_request> http_requests = reduce(packets_stream, recognize_reque
sender auto auth_logic = ... // may invoke 3rd party services
stream<http_request> authenticated_requests = map(http_requests, auth_logic
sender auto request_handler = ... // may use multiple threading resources
stream<http_request> handled_requests = map(authenticated_requests, request
sender auto send_response = ... // again, switches threads
stream<http_request> auto res = map(handled_requests, send_response);
```

For simplicity, the error handling here is done in each of the phases; again this is just a sketch.

We chose this example because some steps (if not all) may switch threads, and may even wait for responses coming over network.

Coming back, the main idea of this section is that we can use computations to represent concurrent structures in which one definition of work is applied many times as different items flow through the structure.

§ 6. Final thoughts and conclusions

§ 6.1. Computations and senders

In § 3.2 Tasks and computations we defined async computation as being exactly as [P2300R2] names *sender object*. It might not have been apparent to the reader why we use a different term and not*senders*. Hopefully, after going through the whole paper, the reader might feel that the name *computation* better describes what can be done with these entities. These entities can describe general computations, from small ones to entire applications. As implementation details, they might be sending values to receivers, but from the user's perspective, they just describe computations. Using the term *sender* doesn't properly describe their usage.

It's also worth mentioning that the term *computation* applies to multiple paradigms. It can be easily used to describe imperative work, it can be well assimilated by functional programmers, it can apply to reactive programming and to all stream-based paradigms; although we haven't talked about it, we can think of computations also in the context of the actor model.

We haven’t explicitly pursued this, but one can infer that computations can be used as a basis for concurrency in multiple programming paradigms.

§ 6.2. Computations vs raw tasks

This paper proved a series of properties for models based on raw tasks and based on computations as defined by [P2300R2]. The reader might have the impression that both models are equivalent. This is far from the truth.

First, there is the low-level semantics, which was mostly ignored by this paper. Probably the main difference is that raw tasks don’t have a guarantee for completion notification. One can build these guarantees on top of tasks, but doing this, will essentially move the model towards [P2300R2]. Raw tasks, when implemented generically, also tend to suffer in terms of performance. More importantly, from a software quality perspective, raw tasks lack proper error handling and cancellation support. All these are aspects, although very important, glanced over by this paper.

Secondly, we arrived at some conclusions for computations that simply do not apply to raw tasks. Computations can represent work chunks that have inner parts that can be executed in different execution contexts. Computations can represent work chunks of arbitrary size: from small units of work to large chunks of works, and even to entire programs. Raw tasks are always bound to one thread, so they cannot simply do all of these.

Theorem 4 (general top-down decomposition), very important from a design perspective applies only to computations and cannot apply to tasks.

Computations (a.k.a., senders) are superior to raw tasks from all these perspectives.

§ 6.3. Computations as abstractions

Allowing top-down decomposition of concurrency, as resulted from Theorem 4 (general top-down decomposition) has a huge impact on the design of concurrent applications. With computations, we have now a proper *abstraction* to be used in concurrent design.

Raw threads and locks are primitives for building concurrent applications, but they are not abstractions. They can be used to build concurrent programs, but they cannot be used to abstract parts of the concurrent system. Moreover, the experience of more than 50 years using these primitives showed us that they are poor primitives as they lead to a lot of safety and performance issues. The software industry needs to abandon the use of raw threads and locks as primitives.

Tasks can also be considered as primitives for building concurrent programs. One can build efficient concurrent programs, and, if the conflicts are correctly set, then the program is safe from a concurrency perspective. Tasks can be better primitives compared to raw threads and locks. But tasks are still not abstractions. One cannot have a task that represents a large part of the concurrent program.

On the other hand, computations are proper abstractions. One can use computations to encode tasks; this makes computations also be primitives for building concurrent programs. Moreover, one can use computations to represent large parts of the concurrent programs and even the whole program.

Let us make an analogy. All the imperative single-threaded programs can be built with `if` and `while` statements. These can be thought of as primitives for imperative programming. But they are not abstractions; functions are proper abstractions. One can encode in a single function large parts of the functionality of a program. And, it can decompose the program with the help of the functions.

Computations are to concurrency what functions are to imperative programming.

6.4. General takeaways

We've shown in this paper that computations can be used as a general mechanism for solving all concurrency problems. We've also shown that computations can be used as a top-down mechanism to describe concurrency; thus it provides a way for doing *structured concurrency*. All these can be made without compromising safety, and ensuring the maximum efficiency (under certain assumptions and ignoring the time spent in scheduling).

Using computations and possibly some higher-level abstractions, the programmer will no longer need to add locks into the code; this has the potential of freeing the C++ world from a lot of pain induced by ad-hoc concurrency.

This may be a bold statement, but after these results, we might dare to say that *computations solve concurrency*.

7. Recommendations

With this paper, the author makes the following recommendations for the C++ standard committee:

1. Work towards adopting as soon as possible the model described in [P2300R2]
2. (minor) Rename *sender* from [P2300R2] into *async computation* (shorter: *computation*), and *receiver* into *async notification handlers*

3. Start working towards providing more concurrency abstractions on top of computations; examples: serializers, pipelines, task graphs, reactive programming, etc.

§ Index

§ Terms defined by this specification

§ References

§ Normative References

[P2300R2]

Michał Dominiak, Lewis Baker, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach. *std::execution*. 4 October 2021. URL: https://wg21.link/p2300r2

§ Informative References

[Böhm-Jacopini]

Corrado Böhm; Giuseppe Jacopini. *Flow Diagrams, Turing Machines and Languages With only Two Formation Rules*. Communication of the ACM, May 1966. URL: http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.119.9119&rep=rep1&type=pdf

[Teodorescu20a]

Lucian Radu Teodorescu. *Refocusing Amdahl's Law*. Overload 157, June 2020. URL: https://accu.org/journals/overload/28/157/overload157.pdf

[Teodorescu20b]

Lucian Radu Teodorescu. *The Global Lockdown of Locks*. Overload 158, August 2020. URL: https://accu.org/journals/overload/28/158/overload158.pdf

[Teodorescu20c]

Lucian Radu Teodorescu. *Concurrency Design Patterns*. Overload 159, October 2020. URL: https://accu.org/journals/overload/28/159/overload159.pdf

[Teodorescu21a]

Lucian Radu Teodorescu. *Composition and Decomposition of Task Systems*. Overload 162, April 2021. URL: https://accu.org/journals/overload/29/162/overload162.pdf

[Teodorescu21b]

Lucian Radu Teodorescu. *Executors: a Change of Perspective*. Overload 165, October 2021. URL: https://accu.org/journals/overload/29/165/overload165.pdf